Reviewer Pack — Minimal Tropical Symplectic Volume and SAT Clause Subset Ent…

Entry b7d9332ccb9b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 23:34:52 UTC

Minimal Tropical Symplectic Volume and SAT Clause Subset Entropy Correlation

Entry ID: `b7d9332ccb9b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-04 23:34:52 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Boolean Satisfiability

Statement:

> For every CNF formula φ with n variables, the minimal tropical symplectic volume (TSV(φ)) of its associated matrix representation is linearly correlated with the entropy of the clause subsets in φ, such that TSV(φ) = Θ(Entropy(φ)).

Rationale (proposer's reasoning):

> Tropical geometry provides a geometric framework for studying computational problems, and symplectic volume captures the geometric complexity of certain polytopes. By relating this to the entropy of clause subsets in SAT formulas, we may expose a novel geometric perspective on the complexity of satisfiability.

Taxonomy category: `TROPICAL_GEOMETRY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `7e8ea192c4e24949`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The correlation between minimal tropical symplectic volume (TSV) and clause subset entropy is considered supported if the Spearman's rank correlation coefficient is >= 0.7, with all seeds showing a positive correlation. Falsification occurs if any seed results in a Spearman's rank correlation coefficient < 0.5 or negative.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `tropical geometry AND Boolean satisfiability AND 'minimal tropical symplectic volume' AND 'SAT clause subset entropy'` - `TSV(CNF formula) AND correlation WITH 'entropy of clause subsets' IN 'Boolean satisfiability' OR 'tropical geometry'` - `entanglement in tropical geometry AND 'Boolean satisfiability problem' AND 'symplectic volume'`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=244.3s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(c == 0 for c in clause):
                continue
            clauses.append(clause)
        return clauses

    def matrix_representation(clauses):
        n = len(clauses[0])
        m = len(clauses)
        A = [[0] * (n + 1) for _ in range(m)]
        for i, clause in enumerate(clauses):
            for j in clause:
                if j > 0:
                    A[i][j - 1] = 1
                else:
                    A[i][-1] += 1
        return A

    def tropical_symplectic_volume(A):
        n = len(A[0]) - 1
        m = len(A)
        volume = 0
```

```
    for i in range(m):
        max_val = -math.inf
        for j in range(n):
            if A[i][j] > max_val:
                max_val = A[i][j]
        volume += max_val
    return volume

def entropy(clauses):
    n = len(clauses[0])
    counts = [0] * (n + 1)
    for clause in clauses:
        for j in clause:
            if j > 0:
                counts[j - 1] += 1
            else:
                counts[-1] += abs(j)
    total = sum(counts)
    entropy = 0
    for count in counts:
        if count > 0:
            p = count / total
            entropy -= p * math.log2(p)
    return entropy

def spearman_correlation(ranks):
    n = len(ranks)
    ranks_x, ranks_y = zip(*ranks)
    x_bar = sum(ranks_x) / n
    y_bar = sum(ranks_y) / n
    numerator = sum((x - x_bar) * (y - y_bar) for x, y in ranks)
    denominator = math.sqrt(sum((x - x_bar)**2 for x in ranks_x)) * math.sqrt(sum((y - y_bar)*
    return numerator / denominator

def rank(data):
    sorted_data = sorted(data)
    rank_dict = {value: idx + 1 for idx, value in enumerate(sorted_data)}
    return [(rank_dict[value], value) for value in data]

n_values = [5, 10, 15, 20, 30, 40]
results = []
for n in n_values:
    clauses = generate_cnf(n)
    A = matrix_representation(clauses)
    tsv = tropical_symplectic_volume(A)
    ent = entropy(clauses)
    results.append((tsv, ent))

ranks_tsv = rank([result[0] for result in results])
ranks_ent = rank([result[1] for result in results])
correlation = spearman_correlation(ranks_tsv)

return {
    "metric_name": "Spearman's Rank Correlation Coefficient",
    "metric_value": correlation,
    "instances_tested": len(results),
```

```
        "n_max": max(n_values),
        "conjecture_holds": correlation >= 0.7,
        "counterexample": "" if correlation >= 0.5 else "Spearman's rank correlation coefficient <
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results) and min(r["metric_value"] for r in result
        print(f"RESULT: FALSIFIED counterexample='Spearman's rank correlation coefficient < 0.5' f
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means that the correlation could not be evaluated. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15027
2	propose	ollama_remote	glm4:latest	0	0	12062
3	preregistration	ollama_remote	glm4:latest	0	0	9587
4	novelty	ollama_remote	glm4:latest	0	0	8648
5	novelty	ollama_remote	glm4:latest	0	0	9008
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17965
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13730
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17523
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20212
10	judge	ollama_remote	glm4:latest	0	0	69568

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 193329 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/b7d9332ccb9b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b7d9332ccb9b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b7d9332ccb9b.tar.gz` (if generated)